

Problem Statement

In compiler testing, we need diverse input programs to stress-test different phases of the compiler — particularly the optimization passes. Writing these by hand is impractical at scale. We asked: *can an LLM serve as an automatic test case generator for LLVM IR, and can those generated cases be used meaningfully in differential testing across optimization levels?*

This sits at the intersection of two things she'll appreciate — compiler internals (IR, optimization passes, code generation) and automated test generation.

What We Built

A pipeline with four stages, each mapping to concepts from compiler theory:

1. **IR Generation** — the LLM is prompted to produce LLVM IR directly, targeting constructs like phi nodes, memory operations, control flow, and arithmetic. Think of it as generating programs at the IR level rather than source level.
2. **Mutation Engine** — seed IR is systematically mutated, similar in spirit to mutation testing. We vary operands, instruction types, and basic block structure to increase test diversity.
3. **Validation Pipeline** — before any mutant enters testing, it goes through LLVM's own toolchain: `llvm-as` checks structural validity, `opt` verifies it survives an optimization pass, and `lli` confirms it can be interpreted. This is the compiler itself acting as the judge.
4. **Differential Testing** — valid mutants are compiled under different optimization levels (`-O0` vs `-O2` etc.) and the outputs are compared. Any divergence is a potential compiler bug.

The pipeline also has a web UI that visualises mutant graphs and shows per-mutant validation status.

Deliverables

- A working end-to-end pipeline from LLM prompt to differential test result
 - A corpus of LLM-generated LLVM IR mutants
 - A visual graph of mutant relationships and outcomes (`graph.html`)
 - A validated backend with a patched `filter_valid.py` handling batch validation state correctly
 - Synopsis and Final Report in `docs/`
-

Results

Out of 67 mutants generated:

- **24 were valid** — passed `llvm-as` , survived `opt` , and executed correctly under `lli`
- **47 were invalid** — failed at one or more stages of the validation pipeline

That's roughly a **36% validity rate** from raw LLM generation, which is actually a useful finding — it tells us the LLM has a partial but imperfect understanding of LLVM IR semantics. The valid 24 were used for differential testing.

A notable engineering issue we hit: a bug in the validation backend was causing all 67 to be reported as invalid due to stale state not being returned correctly in `validate_batch` . After patching it to return existing log entries for already-validated mutants, the true 24/47 split emerged.

Future Improvements

- **Better prompting strategy** — few-shot examples showing valid IR constructs should push the validity rate well above 36%. The LLM knows IR syntax but struggles with semantic correctness (type consistency, SSA form, terminator rules).
- **Targeted mutation operators** — mutations that are provably semantics-preserving (e.g., dead code insertion, instruction reordering within a basic block) so more mutants survive validation.
- **Automated oracle for differential results** — currently divergence detection is heuristic. A proper oracle would distinguish real compiler bugs from undefined behaviour differences.
- **Multi-version testing** — run the valid mutants across LLVM 14, 15, 16 and compare, not just across optimization levels.
- **Feedback loop** — failed mutants can be fed back as negative examples to guide the LLM toward structurally sound IR in future generations.

The key message for her: this is essentially using an LLM as a fuzzer that operates at the IR level, with the compiler's own toolchain as the validity filter. The 36% pass rate is a concrete, measurable result worth discussing.